

Project brief: build an intelligence platform for a financial firm

You have seen a working example: Firm Intelligence, built for the legal sector. It has a FastAPI backend, structured records, Claude-powered summaries, a Chroma vector database fed by Voyage embeddings, a retrieval-augmented question answering endpoint, a tool-using agent and a Streamlit front end.

Your job is to build the same shape of system for a different domain, and to make it your own. You choose the domain from the list below. You may not simply rename "firm" to "bank". The functionality must differ, because the business problem differs.

What you must build

All of the following must be present and working against real, running services. Nothing faked or hardcoded.

1. API (FastAPI)

- A health endpoint.
- Structured records for at least two entity types in your domain, with create, read, update, delete and at least two query filters. Validate input with Pydantic.
- Consistent error handling. Map provider failures to sensible status codes (timeout to 504, rate limit to 429, other upstream failure to 502).
- Configuration from environment variables (API keys, model names, thresholds). No secrets in code.

2. LLM features (Claude)

- A plain summary endpoint for one of your records.
- A streaming version of that summary, so text reaches the client as it is generated.
- A structured analysis endpoint whose output is validated against a Pydantic model of your own design. It must return typed fields, not free text.
- Prompts that keep rules in the system prompt and data in the user message.
- A token estimate endpoint or token counts returned with every generated answer.

3. Vector database (Chroma with Voyage embeddings)

- A persistent Chroma collection.
- A manual index endpoint that embeds your documents and upserts them by id.
- A search-only endpoint that returns the top results with their scores.
- A grounded question answering endpoint. It must cite the documents it used, and it must refuse without calling the LLM when no result clears a relevance floor that you choose and justify.
- At least 8 documents of realistic prose in your domain.

4. Agent (tool use)

- An agent loop with a hard iteration limit and a clear result when the limit is reached.
- A knowledge search tool, plus **at least one more tool of your own** that works on your structured records or computes something (see the domain options for ideas).
- Safe tool execution: unknown tools, missing arguments and tool failures are reported back to the model as errors, not raised as crashes.
- Return the number of tool calls and total token usage with the answer.

5. UI (Streamlit, in its own project with its own environment)

- **Ask:** free text question to the agent, showing the answer, tool calls, tokens and a distinct message if the agent did not complete.
- **Search only:** retrieval without generation, listing every result with title and score. Handle "index not built" differently from an ordinary failure.
- **Streaming summary:** pick a record, watch the text arrive as it streams.
- **One feature of your own design** that fits your domain (a comparison view, a watchlist, an alert triage screen, a chart, anything that uses your API in a way the law firm version did not).

6. Engineering basics

- Tests with the LLM and embedding calls mocked. Cover at least the summary, the refusal rule and the agent loop, including the iteration limit.
 - A `requirements.txt` for each project.
 - A README with the exact commands to run the API and the UI, and the environment variables needed.
-

Choose your domain

Pick one. Each option below gives a real business problem, the data you would model, the documents you would write, some questions your users would ask and a twist that forces you to depart from the law firm version.

A. Financial Markets: trading desk market intelligence

The problem. A sales and trading desk is buried in research notes, central bank statements, economist commentary and desk chat. A trader needs to know quickly what changed, what is relevant to their book and what the house view is.

- **Records:** instruments (rates, FX, credit, equities), desks, house views by asset class.
- **Documents:** morning research notes, central bank meeting summaries, sector commentary, risk limit policies, post-trade reviews.
- **Questions:** "What has the house view on the two year gilt been since the last rate decision?", "Which of our credit desk views conflict with the latest central bank language?"

- **Differ it:** add a tool that computes a simple risk figure from positions (for example duration weighted exposure or a value at risk approximation). Make the streaming summary a "what changed since yesterday" briefing. Make time matter: documents have dates and stale research must be flagged or down-weighted.

B. Retail Banking: complaints and conduct

The problem. A bank receives thousands of customer complaints. Regulators expect root causes to be found, vulnerable customers to be handled properly and outcomes to be consistent.

- **Records:** complaints (product, channel, severity, status), customers (segment, vulnerability flag), products.
- **Documents:** complaint handling policy, regulator guidance, past ombudsman decisions, product terms, redress methodology.
- **Questions:** "How should a late mortgage payment complaint from a vulnerable customer be handled?", "What themes are driving current card complaints?"
- **Differ it:** the structured analysis classifies a complaint into theme, severity and recommended next step. Add a tool that finds similar historic complaints. The refusal rule should be strict, because giving a wrong answer about redress is a conduct risk.

C. Corporate and Investment Banking: credit analysis

The problem. A credit analyst must assess a borrower quickly, drawing on financials, covenant terms, prior credit memos and sector outlooks, then produce a defensible recommendation.

- **Records:** borrowers (sector, rating, leverage, interest cover), facilities, covenants.
- **Documents:** credit memos, sector outlooks, covenant summaries, lending policy, restructuring case studies.
- **Questions:** "Which borrowers are close to breaching an interest cover covenant?", "What does our lending policy say about leveraged loans in cyclical sectors?"
- **Differ it:** add a tool that tests covenant headroom from the numbers. The structured output is a credit assessment with a strengths list, a risks list and a proposed rating with rationale. The streaming summary is a draft credit memo section.

D. Insurance: claims and underwriting

The problem. Underwriters and claims handlers need to apply long, detailed policy wording and guidelines consistently, and to spot claims that look unusual.

- **Records:** policies (line of business, limits, exclusions), claims, brokers.
- **Documents:** underwriting guidelines, policy wordings, loss reports, reinsurance treaties, claims handling manuals.
- **Questions:** "Is flood damage from a burst pipe covered under this commercial property policy?", "Which open claims exceed the retention on our treaty?"

- **Differ it:** add a tool that checks a claim against policy limits and deductibles. The structured output is a coverage opinion with clause references. Refusal matters: when the wording does not support an answer, the system must say so.

E. Asset and Wealth Management: fund research and mandate compliance

The problem. A portfolio manager must decide whether a holding fits a client mandate, and an adviser must explain fund performance and risk in plain language.

- **Records:** funds (strategy, fees, risk rating), portfolios, client mandates with restrictions.
- **Documents:** fund factsheets, manager commentary, investment mandates, ESG policies, regulatory suitability rules.
- **Questions:** "Does this fund breach the client's no tobacco restriction?", "Explain why the emerging market fund underperformed last quarter."
- **Differ it:** add a tool that screens a portfolio against mandate restrictions. Provide a fund comparison view in the UI. Streaming summary is a client-friendly fund update, so tone and plain language become part of your prompt design.

F. Payments and Fintech: fraud and AML operations

The problem. A payments company generates far more fraud and anti money laundering alerts than analysts can review. Analysts need help triaging alerts and finding the right procedure.

- **Records:** alerts (amount, corridor, rule triggered, score), accounts, analysts.
- **Documents:** typology guides, investigation procedures, sanctions guidance, suspicious activity reporting steps, false positive case notes.
- **Questions:** "This alert shows structuring across three accounts, what does the procedure say?", "Which open alerts share a counterparty?"
- **Differ it:** add a tool that links alerts by shared attributes. The structured output is a triage decision (escalate, close, request information) with reasons. Build an alert queue view in the UI. Add a hard rule that the model can never close an alert by itself, it can only recommend.

G. Other (pitch your own)

Other financial services are welcome: mortgages, pensions, private equity, commodities trading, treasury and liquidity, tax advisory, regulatory reporting. To propose one, write half a page that answers:

1. Who is the user, and what decision are they trying to make?
2. What are the two structured entities?
3. What are the eight or more documents, and why is prose the right form for them?
4. What is the extra tool for the agent, and what does it compute or look up?
5. When should the system refuse to answer?

Get the pitch agreed before you start building.

How your version must differ

Whichever domain you choose, your project must show these departures from the law firm version.

1. **Your own data model.** Different entities, fields and filters. No "revenue per lawyer" style calculation copied across.
 2. **Your own extra tool.** The agent has a tool that is not a document search.
 3. **Your own analysis schema.** The structured output has fields specific to your domain.
 4. **Your own refusal rule.** Decide what the system should not answer, and prove it in a test.
 5. **One time-sensitive or risk-sensitive element.** For example document dates, limits and thresholds, or a human-in-the-loop rule.
 6. **One UI feature that the law firm version did not have.**
-

Data rules

- All data is synthetic. Do not use real customer data, real account numbers or confidential material of any employer.
 - Invent firm names, people and figures. Real public concepts (a central bank rate decision, a covenant, a typology) are fine.
 - Write documents the way professionals in the domain write. Terse research notes, formal policy language and case notes are all better than generic paragraphs.
 - At least 5 structured records per entity and at least 8 documents.
-

What you are being judged on

Not polish. Whether:

1. Every feature genuinely calls your real, running API and services, nothing faked.
 2. Failures are handled: provider errors, empty indexes, unknown records, agent limits.
 3. The domain feels real: the data, prompts and refusal rule make sense to someone who works in it.
 4. Your version genuinely differs from the law firm version, following the six departures above.
 5. You can explain, out loud, what every button press does end to end, including what is sent to Voyage, Chroma and Claude.
 6. You can defend your choices: the relevance floor, the iteration limit and the model choice.
-

Design note (required, one page)

Answer briefly:

- Why did you set your relevance floor where you did, and what happened when you tried other values?

- Would you chunk your documents? Why or why not, given their length and shape?
 - What is the worst wrong answer your system could give, and what stops it?
-

Stretch goals

- Split long documents into chunks and store the source document id in the metadata.
 - Filter searches using Chroma metadata (for example by document type or date).
 - Rerank results before sending them to the model.
 - Stream the grounded answer endpoint, not just the summary.
 - Build a small evaluation set of questions with expected source documents and report how often retrieval finds the right one.
 - Add simple authentication and rate limiting to the API.
 - Add a Dockerfile and a compose file that runs both projects.
-

Suggested timeline

Stage	Outcome
Day 1	Domain chosen and pitched, data model designed, documents written
Day 2	API with records, health, CRUD and LLM summary endpoints
Day 3	Chroma indexing, search and grounded answers with a refusal rule
Day 4	Agent with your extra tool, tests for the agent loop
Day 5	Streamlit UI with the three required features and your own
Day 6	Hardening, README, design note and demo rehearsal

Hand in

- The API project and the UI project, each runnable from its README.
- Your synthetic data and documents.
- The one page design note.
- A live demo of at least one question the agent answers using both tools, and one question it correctly refuses.